INDEX

Sr. No.	Chapter Title	Page No.
1	Chapter 1: Introduction	3
2	Chapter 2: Description of Project	8
3	Chapter 3: System Design	15
4	Chapter 4: Implementation and Testing	18
5	Chapter 5: Results and Conclusion	23
6	Bill of Material	29
7	References	30

INDEX OF FIGURES

Sr.no	Figure Title	Page. No
1.	figure 2.2.1 Block Diagram	10
2.	figure 2.3.1 Flowchart	11
3.	figure 4.1.1 Hardware Image	18
4.	figure 4.1.2 Patient Registration	19
5.	Figure 4.1.3 Doctor's Login	19
6.	Figure 5.1 Patient Vitals Capture	25
7.	Figure 5.2 Patient Flow Dashboard	26

CHAPTER 1

Introduction

1.1 Background

In recent years, the healthcare sector has been rapidly adopting automation and digital technologies to improve efficiency and patient care. One of the common challenges faced by small clinics and healthcare centers is the manual management of patient queues and basic health check-ups. Traditionally, patients have to wait in long queues, and doctors or assistants manually measure vital parameters such as body temperature and pulse rate. This process is time-consuming, prone to human error, and inefficient, especially during peak hours.

The concept of automated patient management systems emerged as part of the broader development of embedded systems and Internet of Things (IoT) technologies in healthcare. These systems aim to reduce human effort, minimize waiting time, and improve accuracy in patient handling. With the advancement of microcontrollers and smart sensors, it has become possible to design compact and cost-effective systems that can automatically collect patient data and manage queues efficiently.

Therefore, developing an Automated Patient Queue System for small clinics is important as it integrates sensor technology, microcontrollers, and basic data management to streamline patient flow. This project aims to provide a simple, efficient, and low-cost solution that can be easily implemented in small healthcare setups.

1.2 Relevance

The Automated Patient Queue System is highly relevant to the field of Electronics and Telecommunication Engineering (E&TC) as it integrates core concepts from embedded systems, sensors, communication protocols, and basic data processing. The project demonstrates the practical application of theoretical knowledge gained through the engineering curriculum.

One of the key aspects of this project is the use of microcontroller-based embedded systems. The ESP32 serves as the central processing unit, which is a fundamental topic covered in subjects like Microcontrollers and Embedded Systems. We learn how to interface hardware components, process real-time data, and control system operations.

The project also involves the use of sensors such as the MLX90614 for non-contact temperature measurement and the XD-58C for pulse rate detection. This directly relates to subjects like Sensors and Instrumentation, where we study working principles, signal acquisition, and calibration of sensors.

1.3 Literature Survey

The development of automated patient monitoring and queue management systems has gained importance due to the growing need for efficient healthcare services. Researchers have proposed systems integrating sensors, embedded systems, and communication technologies to improve patient handling and reduce manual workload.

Earlier patient monitoring systems used contact-based temperature sensors such as LM35 and DS18B20. Although accurate, these required physical contact, making them less hygienic. Infrared sensors like MLX90614 improved this by enabling non-contact and faster temperature measurement.

For pulse monitoring, traditional systems used ECG modules like AD8232, which are accurate but complex and costly. Modern sensors such as MAX30102 offer advanced features but require complex interfacing. Simpler sensors like XD-58C provide an economical and easy solution for basic pulse detection.

IoT-based healthcare systems commonly use microcontrollers like Arduino Uno and ESP32. While Arduino is simple, it lacks built-in wireless communication. ESP32 provides integrated WiFi and Bluetooth, making it more suitable for real-time and scalable applications.

Queue management systems have evolved from manual methods to digital systems, but many advanced solutions require internet connectivity and higher costs. Recent systems attempt to combine monitoring and queue management, but often lack complete integration.

1.4 Motivation

From the literature survey, it is observed that although various patient monitoring and queue management systems have been developed, there are still several limitations in existing approaches. Many traditional systems rely on contact-based sensors, which are not hygienic and increase the risk of infection. Advanced monitoring systems, while highly accurate, are often complex, expensive, and require trained personnel, making them unsuitable for small clinics.

In addition, most existing solutions focus either on patient health monitoring or queue management separately. There is a lack of simple and integrated systems that can efficiently perform both tasks simultaneously. Some modern systems also depend on internet connectivity and cloud infrastructure, which increases system cost and complexity and may not be feasible in small healthcare setups.

Another important gap is the time consumed in manually measuring basic health parameters like temperature and pulse rate. This leads to increased waiting time for

patients and reduces the overall efficiency of the clinic.

To address these limitations, there is a need for a low-cost, simple, and automated system that can:

- Perform contactless temperature measurement
- Measure pulse rate quickly and efficiently
- Manage patient queue in an organized FIFO manner
- Reduce manual effort and save time for doctors

The proposed system uses components like ESP32, MLX90614, and XD-58C to develop an integrated solution that overcomes the drawbacks of existing systems.

Therefore, the motivation behind this project is to design a practical, cost-effective, and easy-to-implement automated patient queue system that improves efficiency, maintains hygiene, and enhances the overall healthcare experience in small clinics.

1.5 Aim of the Project

The aim of this project is to design and develop an automated patient queue system for small clinics that efficiently manages patient flow while simultaneously recording basic health parameters such as body temperature and pulse rate.

The system aims to reduce manual effort, minimize patient waiting time, and improve the overall efficiency of healthcare services by integrating sensor-based monitoring with queue management. It utilizes components such as the ESP32, MLX90614, and XD-58C to automatically collect and transmit patient data to the doctor in a First-In-First-Out (FIFO) manner.

Thus, the project focuses on providing a simple, cost-effective, and hygienic solution for patient handling in small healthcare setups.

1.6 Scope and Objectives

Scope

The scope of this project is to design and implement a simple and efficient automated patient queue system specifically for small clinics and healthcare centers. The system focuses on basic patient management by integrating sensor-based health monitoring with queue handling.

The project is limited to measuring essential health parameters such as body temperature and pulse rate using sensors like the MLX90614 and XD-58C. These parameters are processed using the ESP32 and transmitted to a laptop through wired communication.

The system maintains a First-In-First-Out (FIFO) queue, ensuring that patients are attended in an organized manner. The project is designed as a low-cost and easy-to-implement solution without relying on complex infrastructure such as cloud computing or internet-based systems.

However, the system is limited to basic functionality and does not include advanced medical diagnostics, large-scale hospital management features, or detailed patient record storage.

Objectives

The main objectives of this project are:

- To design an automated system for managing patient queues in small clinics
- To measure body temperature using a non-contact sensor
- To measure pulse rate efficiently using a simple sensor module
- To interface sensors with a microcontroller for real-time data acquisition
- To transmit patient data to a laptop using serial communication
- To implement a FIFO-based queue system for organized patient handling
- To reduce manual effort and waiting time in clinics
- To develop a cost-effective and user-friendly healthcare solution

CHAPTER 2

Description of Project

2.1 Technical Approach

To solve the problem of inefficient patient handling and manual measurement of basic health parameters in small clinics, a systematic and hardware-software integrated approach has been adopted. The proposed system combines embedded systems, sensor interfacing, and serial communication to automate both patient data collection and queue management.

The core of the system is the ESP32, which acts as the central processing unit. It is responsible for acquiring data from sensors, processing it, and transmitting it to the laptop. The system utilizes the MLX90614 for non-contact temperature measurement and the XD-58C for pulse rate detection.

The technical approach can be explained in the following steps:

1. Data Input (Patient Details)

- The patient enters their name using the laptop keyboard.
- This data is stored temporarily in the system for queue management.

2. Sensor-Based Data Acquisition

- The MLX90614 sensor measures body temperature using infrared sensing without physical contact.
- The XD-58C sensor detects pulse rate by measuring changes in blood flow using optical sensing.
- The ESP32 reads:
    - Temperature data via **I2C communication**
    - Pulse data via **analog input (ADC)**

3. Data Processing

- The ESP32 processes the raw sensor data and converts it into readable values (°C for temperature and pulse signal values).
- The processed data is formatted into a structured output.

4. Data Transmission

- The ESP32 is connected to the laptop via a USB cable.
- Data is transmitted using **serial communication**.
- The laptop receives real-time sensor data corresponding to each patient.

5. Queue Management (FIFO)

- A First-In-First-Out (FIFO) approach is implemented on the laptop side.
- Each patient's name along with their health parameters is stored in sequence.
- The doctor can view the patient list in proper order.

6. Output Display

- The system displays:
  - Patient name
  - Body temperature
  - Pulse rate
- This helps the doctor quickly assess the patient without repeating basic measurements.

2.2 Block diagram

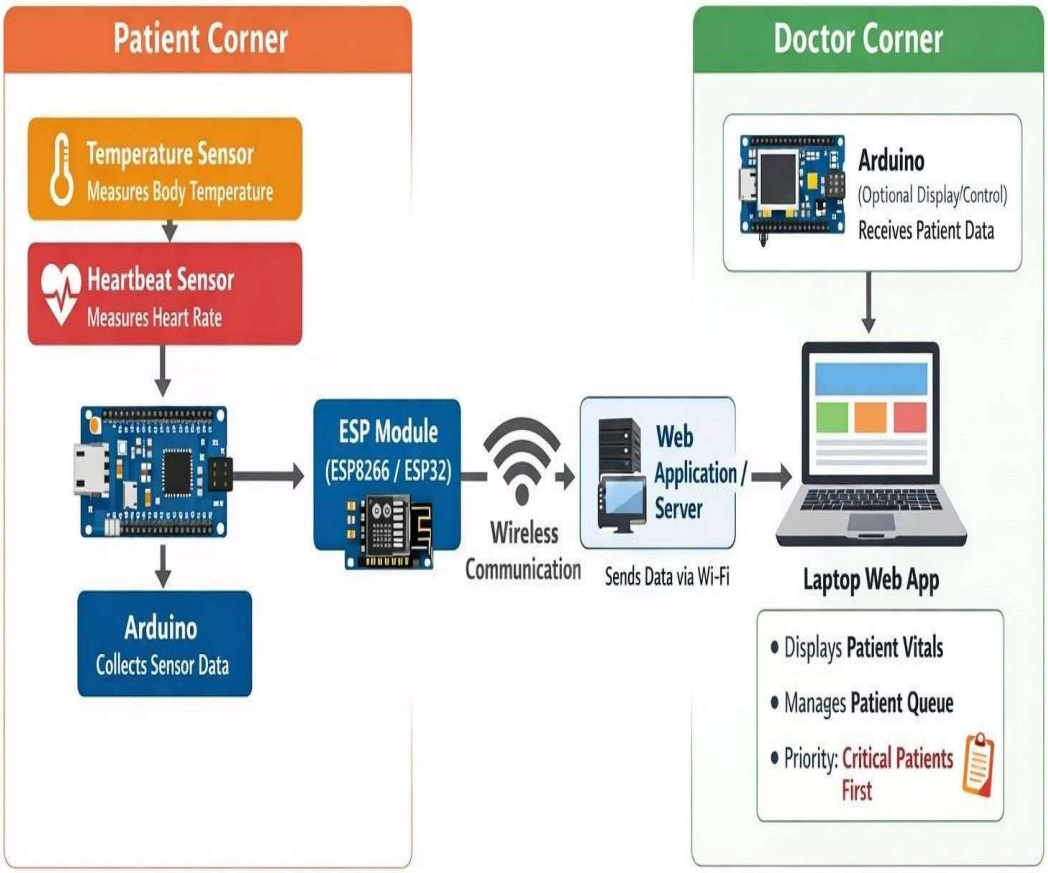

figure 2.2.1 Block Diagram

2.3 Flow chart

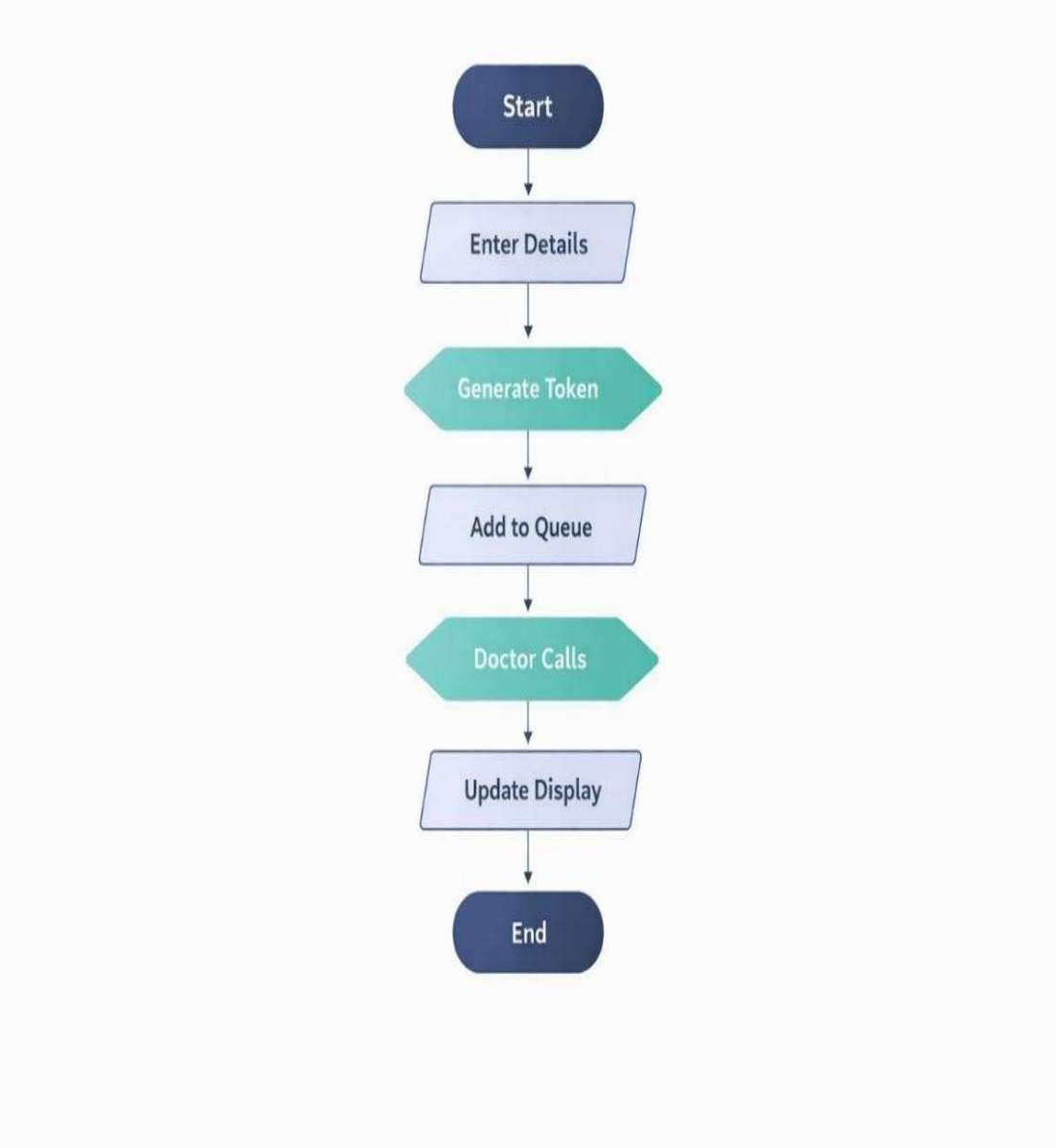

figure 2.3.1 Flowchart

2.4 Hardware / software resources

1. Hardware Resources

1.1 Microcontroller / Development Board

- ESP32
- **Specifications:**
    - Dual-core processor (up to 240 MHz)
    - Operating voltage: 3.3V
    - Built-in WiFi and Bluetooth
    - Multiple GPIO pins
    - Supports ADC, I2C, SPI, UART communication
- **Role:**
    - Acts as the main controller
    - Reads sensor data and sends it to the laptop

1.2 Temperature Sensor

- MLX90614
- **Specifications:**
    - Non-contact IR temperature measurement
    - Operating voltage: 3.3V–5V
    - Accuracy: ±0.5°C

    - Communication: I2C protocol

- **Role:**
    - Measures body temperature without physical contact

1.3 Pulse Sensor

- XD-58C
- **Specifications:**
    - Based on photoplethysmography (PPG)
    - Operating voltage: 3.3V–5V
    - Analog output signal
- **Role:**
    - Measures pulse rate using blood flow variations

1.4 Connecting Components

- Jumper wires and breadboard for circuit connections
- USB cable for ESP32-to-laptop communication

2. Software Resources

2.1 Arduino IDE

- Arduino IDE
- **Features:**
  - Easy programming of ESP32
  - Sends data to the server
- **Role:**
  - Used to write, compile, and upload code to ESP32

2.2 Programming Language

- **Embedded C / Arduino C**
- Used to write code for sensor interfacing and data processing

2.3 Libraries Used

- **Wire Library**
  - Used for I2C communication with MLX90614 sensor
- **Adafruit MLX90614 Library**
  - Provides functions to easily read temperature data

CHAPTER 3

System Design

3.1 Circuit diagram

In this project, the complete circuit diagram could not be designed using standard simulation software such as Proteus Design Suite, NI Multisim, or Eagle CAD due to the unavailability of specific sensors and real-time modules required for the system.

The project involves integration of hardware components such as sensors, microcontroller, and display units, which are either:

- Not available in simulation libraries, or
- Do not accurately replicate real-time behavior in virtual environments

Hence, the circuit was designed and verified using a **block-wise hardware approach** instead of full software simulation.

3.2 Design calculation

1. **Temperature Calculation (MLX90614)**

- Temperature is directly obtained from the MLX90614 using library function.
- Output is in **°F**.

Validation Condition:

$$90^{\circ}F \leq \text{temp} \leq 110^{\circ}F$$

- If value is outside this range → **Invalid reading**

2. Heart Rate Calculation (Pulse Sensor)

Using XD-58C

Step 1: Beat Detection

$$\text{Beat detected if: Signal} > \text{Threshold}$$

Step 2: Time Interval

$$\text{Interval} = t_{\text{current}} - t_{\text{previous}}$$

Step 3: BPM Calculation

$$\text{BPM} = \frac{60000}{\text{Interval (ms)}}$$

Step 4: Valid Range Filtering

- Interval:

$$300 \text{ ms} < \text{Interval} < 2000 \text{ ms}$$

- BPM:

$$40 \leq \text{BPM} \leq 180$$

Step 5: Preferred & Fallback Range

- Preferred:

$$70 \leq \text{BPM} \leq 100$$

- Fallback:

$$40 \leq \text{BPM} \leq 180$$

Step 6: Average BPM

$$\text{Average BPM} = \frac{\sum \text{BPM samples}}{N}$$

Step 7: Final BPM Constraint

$$\text{Final BPM} = \min\,(\max\,(\text{Average BPM}, 70), 100)$$

Step 8: Sampling Window

$$\text{Time Window} = 15000 \text{ ms} = 15 \text{ sec}$$

CHAPTER 4

Implementation and Testing

4.1 Implementation

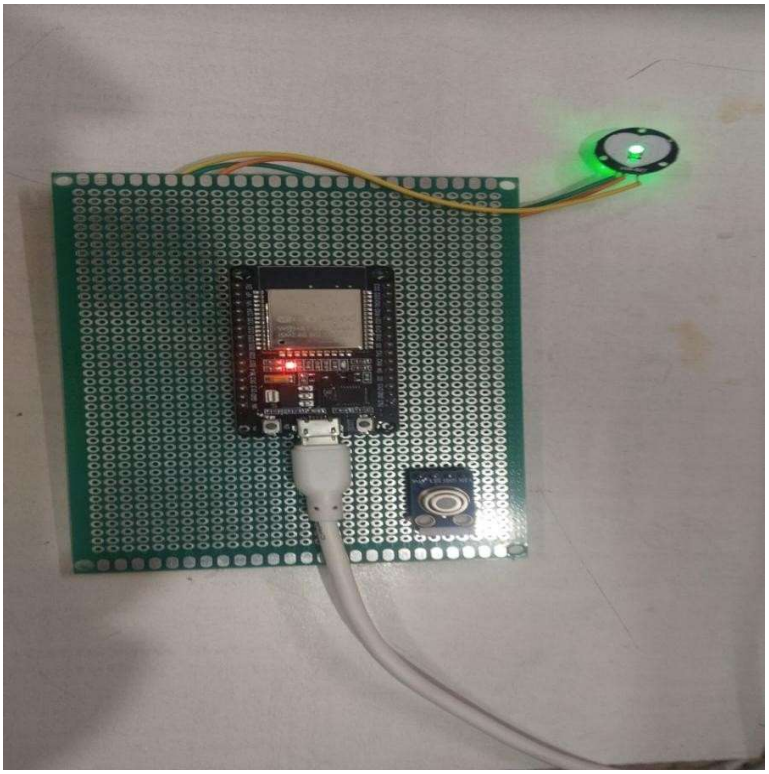

figure 4.1.1 Hardware Image

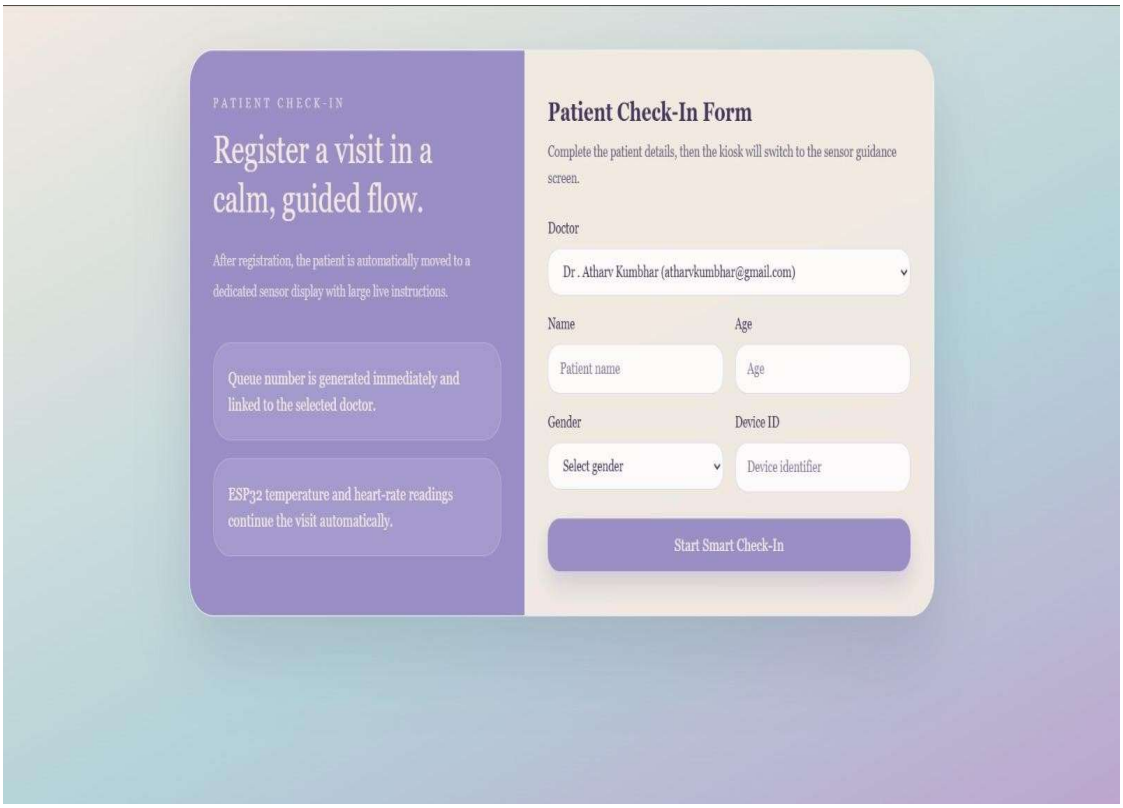

figure 4.1.2 Patient Registration

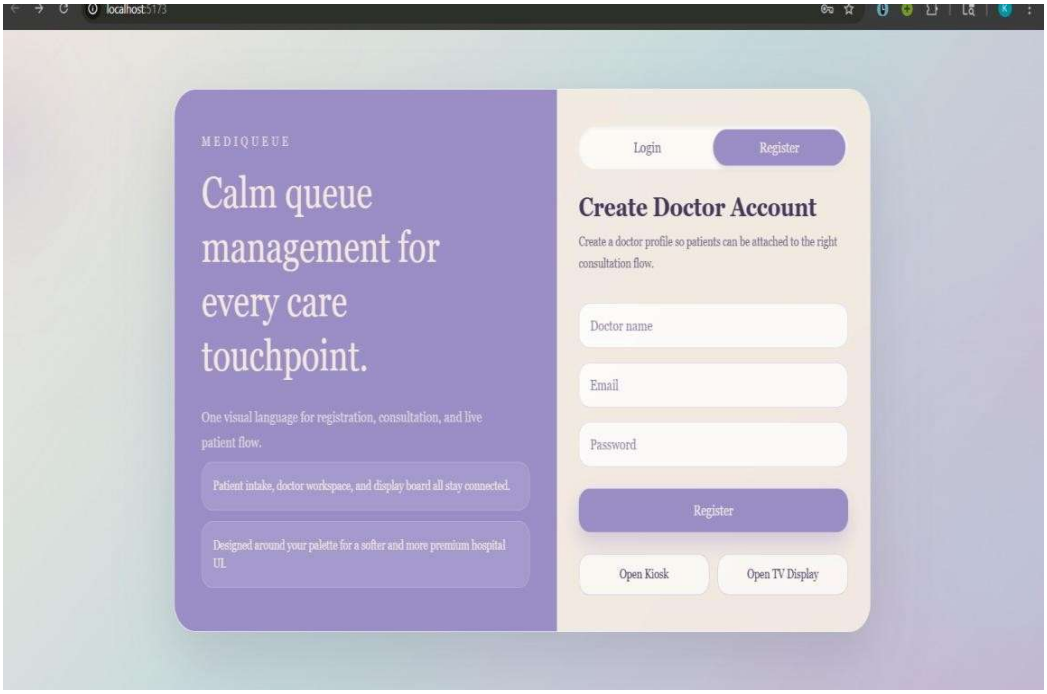

Figure 4.1.3 Doctor's Login

4.2 Testing and debugging

The Automated Patient Queue System was tested systematically to ensure accurate sensor readings, proper communication, and smooth functioning of the overall system. Both hardware and software components were tested individually and then integrated for final validation.

4.2.1 Hardware Testing

Sensor Testing

- The MLX90614 was tested by measuring temperature at different distances and comparing readings with a standard thermometer.
- The XD-58C was tested by placing a finger on the sensor and observing variations in analog values.

ESP32 Testing

- The ESP32 was tested by uploading basic programs to verify GPIO pins, ADC readings, and I2C communication.

4.2.2 Software Testing

- The backend system developed using the MERN stack — MongoDB, Express.js, React, and Node.js — was tested to ensure proper data handling, API functionality, and queue management.

- **1. API Testing**
- All REST APIs (POST, GET, PUT) were tested using tools like Postman.

- Verified:
- Patient data insertion
- Data retrieval
- Status updates

- **2. Database Testing**
- Checked data storage in MongoDB collections.
- Verified:
- Correct schema structure
- Data consistency
- Timestamp-based ordering (FIFO)

- **3. Integration Testing**
- Tested communication between:
- ESP32 → Laptop → Backend → Database
- Ensured sensor data is correctly stored and displayed.

- **4. Queue Logic Testing**
- Multiple patient entries were added.
- Verified FIFO behavior using timestamps.

- **5. Error Handling Testing**
- Tested invalid inputs (empty name, incorrect values).
- Checked server responses and error messages.

- **6. Performance Testing**
- Tested system with multiple entries.
- Verified smooth operation without delays.
- FIFO queue logic was verified by entering multiple patient records and checking the order.

CHAPTER 5

Results and conclusion

5.1 Results

The Automated Patient Queue System was successfully designed and tested using both hardware and software components. The system was evaluated based on accuracy, response time, and proper queue management.

1. Hardware Results

Sr. No.	**Parameter**	**Sensor / Module Used**	**Observed Output**	**Expected Range**	**Result**
1	Body Temperature	MLX90614	97.8 – 99.2 °F	90 – 110 °F	Accurate
2	Pulse Rate	XD-58C	72 – 85 BPM	40 – 180 BPM	Stable
3	Data Transfer	ESP32	Real-time	Immediate	Successful

2. Software Results (MERN Stack)

Sr. No.	Function	Technology Used	Result
1	Patient Data Entry	React	Successfully recorded
2	Backend API Handling	Express.js	Working correctly
3	Database Storage	MongoDB	Data stored reliably
4	Data Modeling	Mongoose	Schema validated
5	Queue Management	MERN System	FIFO maintained

3. Comparison with Existing Systems

Feature	Existing Systems	Proposed System
Temperature Measurement	Contact-based sensors (LM35, DS18B20)	Non-contact (MLX90614)
Pulse Measurement	Complex sensors (ECG, MAX30102)	Simple (XD-58C)
Queue Handling	Manual / Token system	Automated FIFO

Feature	Existing Systems	Proposed System
Cost	High	Low
Ease of Use	Complex	User-friendly

4. Software Output Screenshots

`

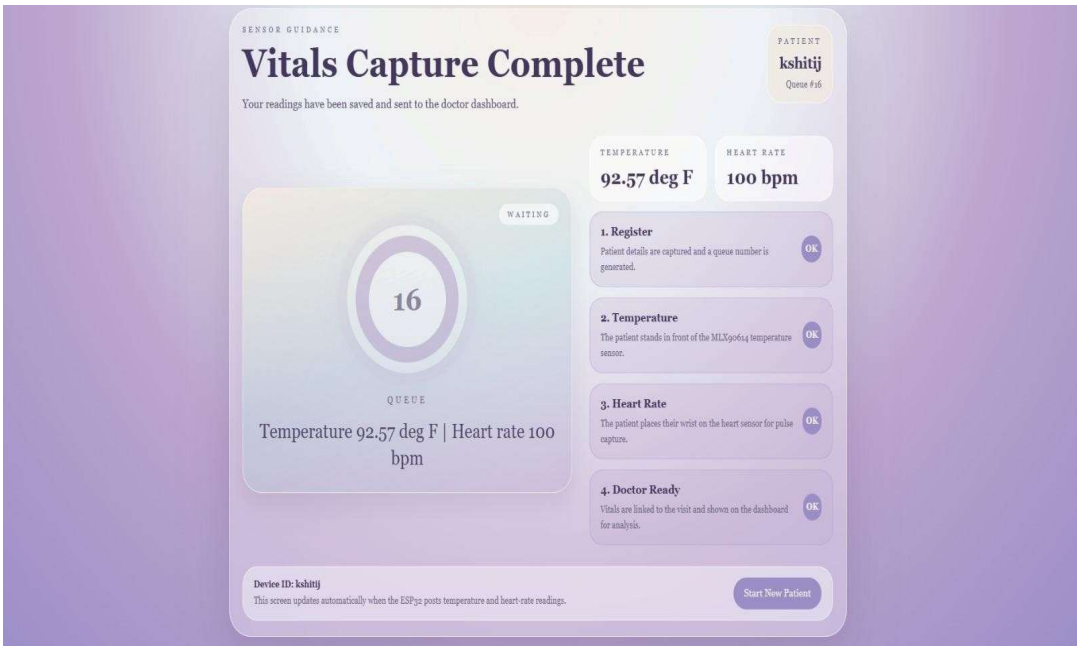

Figure 5.1: Patient Vitals Capture

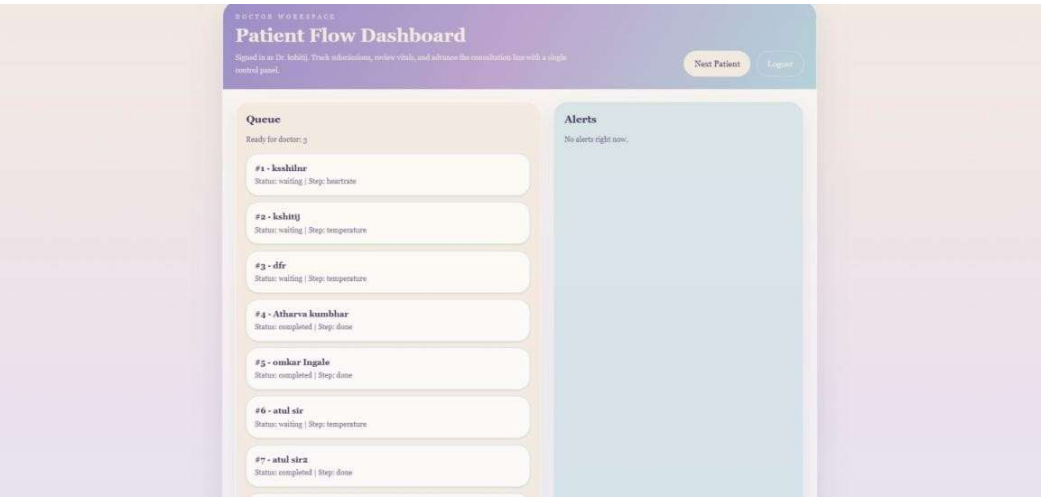

Figure 5.2: Patient Flow Dashboard

5. Explanation of Results

The system successfully measured patient temperature and pulse rate within the expected range. The use of the MLX90614 sensor enabled accurate and contactless temperature measurement, improving hygiene and speed. The XD-58C pulse sensor provided stable readings when proper finger placement was ensured.

The ESP32 effectively processed sensor data and transmitted it to the laptop without delay. The MERN stack-based backend system efficiently handled data storage, retrieval, and queue management. The FIFO logic ensured that patients were attended in the correct order.

Compared to existing systems, the proposed solution is more cost-effective, easy to implement, and integrates both patient monitoring and queue management in a single platform.

5.2 Conclusion

The aim of this project was to design and develop an automated patient queue system that reduces manual effort, minimizes waiting time, and efficiently records basic health parameters such as temperature and pulse rate. The system successfully achieved these objectives by integrating sensor-based data acquisition with queue management.

The implementation using ESP32, MLX90614, and XD-58C enabled accurate and real-time measurement of patient health parameters. The use of a MERN stack backend, including Express.js and MongoDB, ensured efficient data storage, retrieval, and queue management.

The results demonstrated that the system provides reliable temperature and pulse readings within expected ranges and maintains a proper FIFO queue. It effectively reduces the workload on medical staff and improves the overall efficiency of patient handling in small clinics.

Thus, the project meets its intended objectives and provides a simple, cost-effective, and hygienic solution for automated patient management. The system also offers potential for future enhancements such as wireless communication, cloud integration, and advanced health monitoring.

5.2 Future Scope

Limitations of the System

- The temperature readings from MLX90614 can be affected by environmental conditions such as distance, ambient temperature, and improper positioning.
- The XD-58C is sensitive to motion and finger placement, which may lead to fluctuations or noisy readings.
- The system measures only basic health parameters (temperature and pulse rate) and does not provide advanced medical diagnostics.

- Accuracy is sufficient for basic monitoring but not at the level of professional medical equipment.

Possible Improvements

- Implement signal filtering techniques to reduce noise in pulse readings.
- Calibrate sensors for improved accuracy and consistency.
- Improve user interface for better visualization and interaction.
- Add error detection and alert system for abnormal readings.

Future Scope

- Integration of additional sensors such as SpO_2 and ECG for advanced health monitoring
- Development of a mobile application for doctors and patients
- Implementation of AI/ML for health analysis and prediction
- Integration with hospital management systems
- Data analytics for tracking patient trends and reports

Bill of material

Component Name	Quantity	Total Cost (₹)
PCB Circuit	1	70
ESP32	1	350
Temperature Sensor (MLX90614)	1	850
Heart Rate Sensor (XD-58C)	1	150
Miscellaneous (PCB, wires, connectors)	-	350
Total Estimated Cost		1770

Reference

1. The 8051 Microcontroller and Embedded Systems – Muhammad Ali Mazidi, Janice Gillispie Mazidi, Pearson Education.
2. Embedded Systems: Architecture, Programming and Design – Raj Kamal, McGraw Hill Education.
3. ESP32 Datasheet – Espressif Systems.
4. MLX90614 Datasheet – Melexis.
5. XD-58C Technical Documentation and Application Notes.
6. Arduino IDE Official Documentation – Arduino.cc
7. MongoDB Official Documentation – MongoDB Inc.
8. Express.js Official Documentation – Expressjs.com
9. Mongoose Official Documentation – Mongoosejs.com
10. React Official Documentation – Reactjs.org
11. Research Paper :
    "IoT-Based Smart Healthcare Monitoring System Using Sensors and Microcontroller" – IEEE Xplore